INTERAÇÃO COM MODELOS DE LARGA ESCALA

Trabalho Prático #1

From Raw Detections to Real Intelligence

Cotação: 3,5 valores / 20

Ano Letivo 2025/2026

1. Contexto e Motivação

As lojas de retalho modernas estão equipadas com sistemas de visão computacional que geram continuamente eventos de deteção. Cada vez que uma pessoa entra numa zona, permanece ou a abandona, é registado um evento com timestamp, identificador de zona e atributos demográficos agregados e anónimos. Ao longo de uma semana de operação, uma única loja pode gerar centenas de milhares de eventos.

O desafio fundamental destes sistemas é que os dados chegam sem identidade: o sistema sabe que alguém entrou na zona de frescos às 14h32, mas não sabe se é a mesma pessoa que esteve nos produtos de limpeza às 14h28. O percurso individual (a informação mais valiosa para compreender o comportamento do cliente) tem de ser reconstruído a partir deste stream de eventos anónimos e intercalados.

Este trabalho tem duas fases complementares. Na primeira, o aluno resolve um problema algorítmico clássico: reconstruir trajetórias individuais a partir de um stream de eventos sem identidade, usando heurísticas de associação temporal e de atributos. Na segunda, usa um LLM local para transformar as trajectórias reconstruídas em inteligência operacional útil para o gestor de loja: insights acionáveis, anomalias detectadas, e um briefing semanal automático em linguagem natural.

O objetivo final é um pipeline que recebe dados brutos de detecção e produz, sem intervenção humana, um documento que um gestor de loja pode ler segunda-feira de manhã e sobre o qual pode agir imediatamente.

2. Descrição do Problema

2.1 O Cenário

Uma loja de retalho está equipada com um sistema de visão computacional que monitoriza 23 zonas. O sistema produz três tipos de eventos para cada pessoa detetada:

- entry: a pessoa entra numa zona. Regista o instante exato.
- exit: a pessoa sai de uma zona. Regista o instante exato.
- linger: a pessoa permanece numa zona. Regista a duração total de permanência em segundos. Este evento é gerado entre o entry e o exit correspondentes.

O dataset cobre 7 dias consecutivos de operação (segunda a domingo), com horário das 09h00 às 21h00, e contém aproximadamente 250.000 eventos.

2.2 Os Dados

O dataset é fornecido num único ficheiro CSV («events.csv») com os seguintes campos:

event_id	timestamp	zone_id	event_type	duration_s	gender	age_range
e_008167	2025-03-10 09:00:32	Z_E2	entry	0	M	adult
e_008168	2025-03-10 09:00:42	Z_E2	exit	0	M	adult

event_id	timestamp	zone_id	event_type	duration_s	gender	age_range
e_008169	2025-03-10 09:01:00	Z_C1	entry	0	M	adult
e_008170	2025-03-10 09:01:05	Z_C1	linger	23	M	adult
e_008171	2025-03-10 09:01:23	Z_C1	exit	0	M	adult
e_002940	2025-03-10 09:02:58	Z_E2	entry	0	F	middle_aged
...	...	...	...	...	...	...

Onde:

- event_id: identificador único do evento.
- timestamp: data e hora do evento no formato YYYY-MM-DD HH:MM:SS.
- zone_id: zona da loja onde o evento ocorreu (e.g., Z_N5, Z_CK, Z_E1).
- event_type: tipo de evento — entry, exit, ou linger.
- duration_s: duração em segundos. Relevante apenas para eventos linger (0 para entry/exit).
- gender: género estimado (M ou F). Sujeito a erros de classificação (~8%).
- age_range: faixa etária estimada (child, teenager, young_adult, adult, middle_aged, senior). Sujeita a erros (~12%).

A loja tem 23 zonas identificadas por prefixo:

Prefixo	Zonas	Tipo sugerido	Descrição
Z_E	Z_E1, Z_E2	Entradas / Saídas	Portas de acesso principal à loja
Z_C	Z_C1, Z_C2, Z_C3	Caixas registadoras	Pontos de pagamento e checkout
Z_CK	Z_CK	Checkout geral	Área de saída após pagamento
Z_N	Z_N1 … Z_N10	Corredores / Navegação	Vias de trânsito entre seções
Z_S	Z_S1 … Z_S7	Seções de produto	Áreas de exposição e venda

Observação crítica: o dataset não tem person_id. Múltiplas pessoas estão presentes na loja em simultâneo — os seus eventos estão intercalados no stream por ordem cronológica. A reconstrução de trajectórias é o problema central da Fase 1.

2.3 Arquitetura Geral do Pipeline

O aluno deve construir um pipeline composto por quatro módulos independentes e executáveis em seqüência:

```
events.csv
    ↓
[1] stitcher.py          →  journeys.csv
```

```
    ↓
[2] analytics.py          →  metrics.json
    ↓
[3] insights.py           →  insights.json
    ↓
[4] report.py             →  weekly_report.md
```

Cada módulo deve ser executável de forma independente, recebendo o seu input via argumento de linha de comandos e produzindo o seu output num ficheiro. Não devem existir dependências implícitas entre módulos para além dos ficheiros de input/output definidos.

3. Fase 1 - Reconstrução de Trajectórias

3.1 O Problema

Dado o stream de eventos sem identidade, reconstruir quais eventos pertencem à mesma pessoa, atribuindo a cada grupo de eventos um person_id sintético. Cada trajectória reconstruída deve representar a visita completa de uma pessoa à loja: desde a entrada (evento entry numa zona Z_E) até à saída (evento exit numa zona Z_E ou Z_CK), passando por todas as zonas intermédias visitadas.

O problema é formalmente equivalente ao problema de data association em tracking multi-objeto: dado um conjunto de observações no tempo, associar cada observação ao objeto que a gerou, sem nunca observar os objetos diretamente.

3.2 Heurísticas de Associação

O aluno deve conceber e implementar um algoritmo de stitching. A abordagem base deve considerar obrigatoriamente os seguintes critérios:

- Consistência temporal: uma pessoa não pode estar em duas zonas em simultâneo. Se dois eventos entry ocorrem na mesma zona no mesmo segundo com os mesmos atributos, são pessoas diferentes.
- Plausibilidade de transição: o tempo entre o exit de uma zona e o entry da zona seguinte deve ser plausível. Transições com gap temporal superior a 5 minutos sem deteção podem indicar fim de trajectória.
- Consistência de atributos: eventos que fazem parte da mesma trajectória devem ter gender e age_range consistentes — uma pessoa não muda de género a meio da visita. Porém, pode haver deteções com erros devido à dificuldade de se usar visão computacional “in-the-wild”.
- Estrutura entry→linger→exit: cada visita a uma zona tende a seguir esta seqüência. A exploração desta estrutura pode informar a associação.

O aluno é livre de propor heurísticas adicionais, mas deve justificá-las no relatório com base nos padrões observados nos dados.

3.3 Output da Fase 1

Um ficheiro journeys.csv com o seguinte schema:

```
person_id    — identificador sintético (e.g. P_0001)
```

```
zone_id      — zona visitada
entry_time   — timestamp de entrada na zona
exit_time    — timestamp de saída da zona
dwell_s      — tempo de permanência em segundos
gender       — género da pessoa
age_range    — faixa etária da pessoa
visit_date   — data da visita
hour_of_day  — hora de entrada (inteiro 0-23)
```

Cada linha representa a visita de uma pessoa a uma zona. Uma pessoa que visitou 8 zonas gera 8 linhas no ficheiro de saída.

3.4 Avaliação da Fase 1

O aluno deve implementar e reportar as seguintes métricas de qualidade da reconstrução. Estas métricas são calculadas automaticamente pelo harness de avaliação (Secção 5).

- Consistência: percentagem de trajectórias onde nenhuma pessoa está em duas zonas em simultâneo. Qualquer violação indica um erro no algoritmo.
- Cobertura: percentagem de eventos do dataset original que foram atribuídos a alguma trajectória (vs. eventos descartados).
- Completude: percentagem de trajectórias com início numa zona de entrada (Z_E) e fim numa zona de saída (Z_E ou Z_CK).
- Plausibilidade temporal: distribuição dos gaps entre zonas consecutivas na mesma trajectória. O aluno deve argumentar que a distribuição é plausível para uma loja de retalho.

4. Fase 2 - Retail Intelligence com LLM

4.1 Princípio de Separação

Regra fundamental desta fase: a LLM nunca acede diretamente ao CSV nem executa cálculos sobre os dados em bruto. Recebe apenas métricas pré-calculadas em formato JSON. Toda a computação sobre os dados é feita em Python. A LLM é responsável exclusivamente pela síntese e linguagem natural. Esta separação é uma decisão arquitetural que o aluno deve defender no relatório.

4.2 Pipeline Analítico (Python)

Antes de invocar qualquer LLM, o aluno deve calcular as seguintes métricas a partir do journeys.csv:

- Métricas de tráfego: número de visitantes únicos por dia e por hora; distribuição de afluência ao longo da semana; tempo médio de visita à loja.
- Métricas por zona: tráfego por zona; dwell time médio (apenas eventos linger); taxa de paragem (visitantes com linger / visitantes totais); seqüências de zonas mais frequentes (top-10).
- Funil de cliente: de todos os visitantes que entraram, quantos chegaram a cada zona; taxa de conversão para caixa; perfil dos visitantes que não chegaram à caixa.
- Segmentação demográfica: distribuição de género e faixa etária por hora do dia; dwell time médio por segmento e por zona.

- Deteção de anomalias: para cada zona e cada hora, calcular média e desvio padrão do tráfego nos primeiros 6 dias; identificar horas/zonas onde o dia 7 se desvia mais de 2σ da média.

O output deste pipeline é um ficheiro metrics.json — um dicionário estruturado com todas as métricas calculadas, que serve de input exclusivo ao LLM.

4.3 LLM Insight Engine

O aluno deve construir um sistema de prompts que recebe metrics.json e gera insights acionáveis em português. O LLM deve retornar JSON estruturado com o seguinte schema:

```
{
  "insights": [
    {
      "id": "INS_001",
      "categoria": "trafego|zona|funil|anomalia|demografico",
      "titulo": "frase curta que resume o insight",
      "observacao": "o que os dados mostram: factos, números concretos",
      "implicacao": "o que isto significa operacionalmente",
      "recomendacao": "ação concreta que o gestor pode tomar",
      "urgencia": "imediata|esta_semana|proximo_mes",
      "confianca": 0.0
    }
  ],
  "resumo_executivo": "3 bullets com os insights mais importantes"
}
```

O aluno deve implementar e comparar obrigatoriamente duas estratégias de prompting:

- Estratégia A - zero-shot: instrução direta com o schema de output e os dados.
- Estratégia B - few-shot: exemplos de insights bons e maus antes do pedido (ver tabela abaixo).

O que distingue um bom insight de um mau insight:

Mau insight (genérico, vago)	Bom insight (específico, acionável)
A zona de frescos teve bastante tráfego.	A zona Z_S3 teve 847 visitantes na quinta-feira, 31% acima da média semanal.
Recomenda-se melhorar o atendimento ao cliente.	Às quartas entre 12h–13h, o tempo médio de espera em Z_C2 foi 4,2 minutos — abrir uma terceira caixa neste slot reduziria o tempo para < 2 minutos com base no volume observado.
Houve uma anomalia na loja.	No domingo às 16h, Z_N4 teve 0 visitantes contra uma média de 23 — possível obstrução ou problema de sinalização.

O relatório deve comparar quantitativamente os resultados das duas estratégias sobre um conjunto de avaliação definido pelo aluno.

4.4 Report Semanal

O último componente é a geração automática de um briefing semanal em português, estruturado para ser lido por um gestor de loja sem formação técnica. O report deve conter as seguintes secções:

1. Resumo executivo (máx. 150 palavras): os 3 factos mais importantes da semana. Linguagem direta, números concretos, sem jargão técnico.
2. Performance de tráfego: afluência da semana, padrões de hora de pico, dias mais e menos movimentados.
3. Análise de zonas: top 3 com melhor e pior performance; para cada zona problemática: hipótese de causa e recomendação.
4. Funil de clientes: da entrada até à caixa; onde se perde tráfego; perfil dos clientes que não chegam à caixa.
5. Anomalias da semana: eventos fora do padrão; para cada anomalia: descrição, magnitude, possível causa, ação recomendada.
6. Recomendações para a próxima semana: máximo 5 ações concretas, ordenadas por urgência, cada uma específica o suficiente para ser executável sem interpretação adicional.

Formato de entrega do report: Markdown renderízável. O aluno pode opcionalmente converter para PDF.

O aluno pode tomar decisões de alteração do report semanal, desde que justificadas no seu relatório.

5. Harness de Avaliação

O aluno deve construir um harness de avaliação que testa o pipeline de ponta a ponta. Este harness deve ser executável pelo professor com um único comando:

```
python evaluate.py --data events_validation.csv --output evaluation_report.json
```

O docente fornecerá events_validation.csv no momento da entrega. Isto é, um dataset do mesmo formato mas com anomalias injetadas conhecidas. O harness não deve estar otimizado para o dataset de treino.

Métricas calculadas automaticamente pelo harness:

- Consistência: % de trajectórias sem sobreposição temporal (deve ser 100%).
- Cobertura: % de eventos do dataset original atribuídos a alguma trajectória.
- Completude: % de trajectórias com entrada e saída em Z_E ou Z_CK.
- Deteção de anomalias: % das anomalias injetadas corretamente identificadas nos insights.
- Precisão numérica: % de valores numéricos nos insights verificáveis nos dados calculados.
- Ausência de alucinação: % de afirmações factuais no report verificáveis no metrics.json.

6. Entregáveis

6.1 Código

Repositório com a seguinte estrutura:

```
tp1/
├── README.md              — instruções de instalação e execução
├── requirements.txt       — dependências exatas com versões
├── data/
│   └── events.csv
├── src/
│   ├── stitcher.py
│   ├── analytics.py
│   ├── insights.py
│   └── report.py
│   └── *.py
├── prompts/
│   └── *.txt              — todos os prompts, versionados
├── output/
│   ├── journeys.csv
│   ├── metrics.json
│   ├── insights.json
│   └── weekly_report.md
└── evaluate.py
```

O pipeline completo deve correr com:

```
pip install -r requirements.txt
ollama pull llama3.1:8b (ou outro modelo utilizado!)
python src/stitcher.py  --input data/events.csv        --output
output/journeys.csv
python src/analytics.py --input output/journeys.csv     --output
output/metrics.json
python src/insights.py  --input output/metrics.json     --output
output/insights.json
python src/report.py    --input output/insights.json    --output
output/weekly_report.md
```

6.2 Relatório Técnico

O relatório deve ter entre 6 e 10 páginas (excluindo anexos) e incluir obrigatoriamente:

7. Introdução: descrição do problema, dificuldades identificadas no dataset, decisão de arquitetura (separação Python/LLM).
8. Algoritmo de stitching: descrição detalhada das heurísticas, justificação de cada escolha, métricas de qualidade obtidas, análise de casos de falha.
9. Pipeline analítico: métricas calculadas, decisões sobre o que incluir e excluir, formato do metrics.json.
10. Engenharia de prompts: prompts completos de cada estratégia, comparação quantitativa A vs. B, exemplos de bons e maus outputs, iterações realizadas.
11. Avaliação: resultados do harness no dataset de treino, análise de falhas, limitações honestas do sistema.
12. Conclusão: o que funcionou, o que não funcionou, o que faria de diferente.

Anexos (não contam para o limite de páginas): todos os prompts usados; exemplo completo de metrics.json; exemplo completo de insights.json; o report semanal gerado.

6.3 Defesa Oral

No caso de existir defesa oral, esta terá 10 minutos:

- 4 minutos: o aluno explica o algoritmo de stitching e mostra as métricas de qualidade.
- 2 minutos: o aluno executa o pipeline ao vivo sobre o dataset de validação fornecido no momento da defesa.
- 4 minutos: o professor faz perguntas sobre decisões técnicas. Porquê este threshold? Porquê este prompt e não outro? O que acontece se a anomalia for mais subtil?

Nota de avaliação qualitativa: um sistema que funciona e cujos relatórios de falha são honestos e detalhados vale mais do que um sistema que aparentemente funciona mas cujo relatório não identifica nenhuma limitação. A capacidade de analisar criticamente o próprio trabalho é avaliada explicitamente.

7. Orientações Técnicas

- Modelos LLM: qualquer modelo que corra localmente via Ollama ou outro package é permitido. Os modelos recomendados são llama3.1:8b, mistral:7b e novas versões. O aluno deve indicar no README qual o modelo usado.
- Performance do stitching: o algoritmo deve completar em menos de 5 minutos num laptop moderno com o dataset completo. Algoritmos de complexidade $O(n^2)$ sem otimização provavelmente excedem este limite.
- Reprodutibilidade: qualquer componente com aleatoriedade deve ter seed fixo. O harness de avaliação deve usar temperature=0 para garantir resultados reprodutíveis.
- Cenários de teste próprios: antes de receber o dataset de validação, o aluno deve criar os seus próprios cenários injetando anomalias conhecidas no dataset original. Esta prática deve estar documentada no relatório.

9. Regras e Integridade Académica

- Este é um projeto individual. Todo o código deve ser escrito pelo aluno.
- A utilização de ferramentas de geração de código por IA (ChatGPT, GitHub Copilot, Claude, etc.) é fundamental, mas a defesa oral testará a compreensão de cada linha de código. Se o aluno não conseguir explicar a sua própria solução, a nota refletirá isso.
- Caso exista, a defesa oral é obrigatória. A não comparecimento resulta na nota zero para a totalidade do projeto.
- O programa deve executar sem erros. Projetos que terminem em erro durante a demonstração receberão no máximo 45% da cotação.

Bom trabalho. A inteligência começa na qualidade dos dados que lhe forneces.